WordPress Hosting on AWS EC2 with RDS (LAMP Stack)

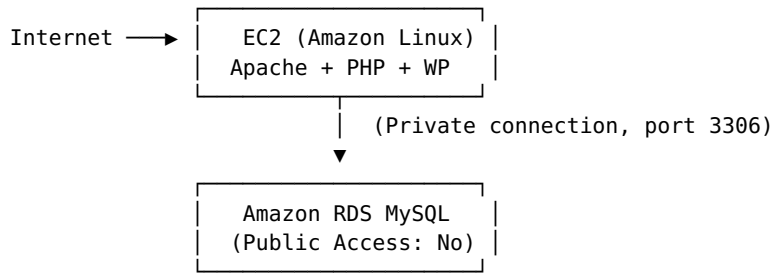
Project Overview

This project demonstrates deploying a production-style WordPress website on AWS using a **decoupled architecture** — the web server (Apache/PHP) runs on an EC2 instance while the database (MySQL) runs on a separate, private Amazon RDS instance. The setup also includes a High Availability extension using an Application Load Balancer (ALB) and an Auto Scaling Group (ASG).

Stack Used: Linux (Amazon Linux 2023) + Apache + MySQL (RDS) + PHP → **LAMP**

Live Output: WordPress blog running on EC2, publicly reachable both via the instance's public IP and via the ALB DNS name.

Architecture Diagram (Conceptual)




```
   High-Availability extension:
   Internet → ALB (Wordpress-ALB) → Target Group (WP-TG) → ASG
(wordpress-ASG) → EC2 (from WordpressAMI) → RDS MySQL
```

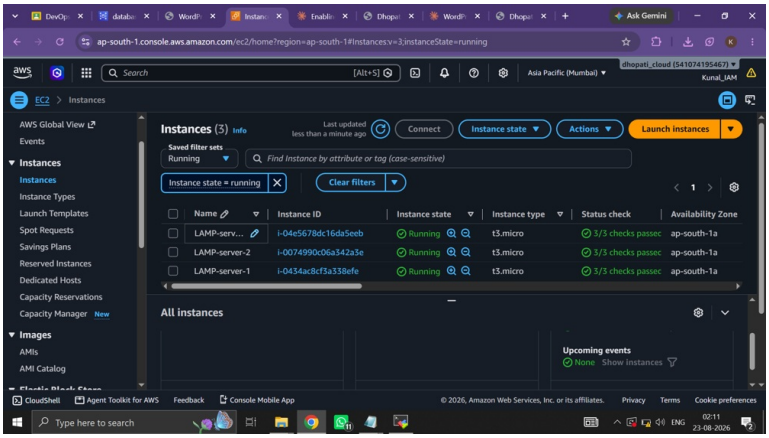
Key AWS Services Used

Service	Purpose
Amazon EC2	Hosts Apache web server + WordPress application
Amazon RDS (MySQL)	Managed, private database backend
Security Groups	Controls network access (HTTP, SSH, MySQL)
Application Load Balancer (ALB)	Distributes traffic across multiple EC2 instances
Auto Scaling Group (ASG)	Maintains desired instance count using a Launch Template
AMI	Reusable image of the configured WordPress server

---

1. Compute — EC2 Instances

3 EC2 instances (Amazon Linux 2023, t3.micro) launched and running, all passing 3/3 status checks.



EC2 Instances Running

Step 1: Launch EC2 Instance

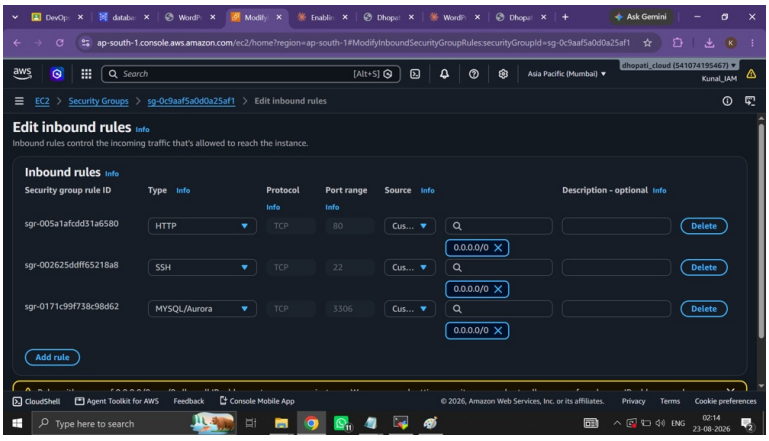
- Launched an **Amazon Linux 2023** EC2 instance.
- Configured a Security Group to allow HTTP (80) and SSH (22).

---

2. Security Group

Rule	Port	Source
HTTP	80	0.0.0.0/0
SSH	22	0.0.0.0/0
MySQL/Aurora	3306	0.0.0.0/0

> ⚠ **Note:** Rules are kept broad (0.0.0.0/0) here for a learning/demo environment. In a production setup, SSH and MySQL access should be restricted to specific IP ranges or a Bastion/internal security group instead of the open internet.



Security Group Inbound Rules

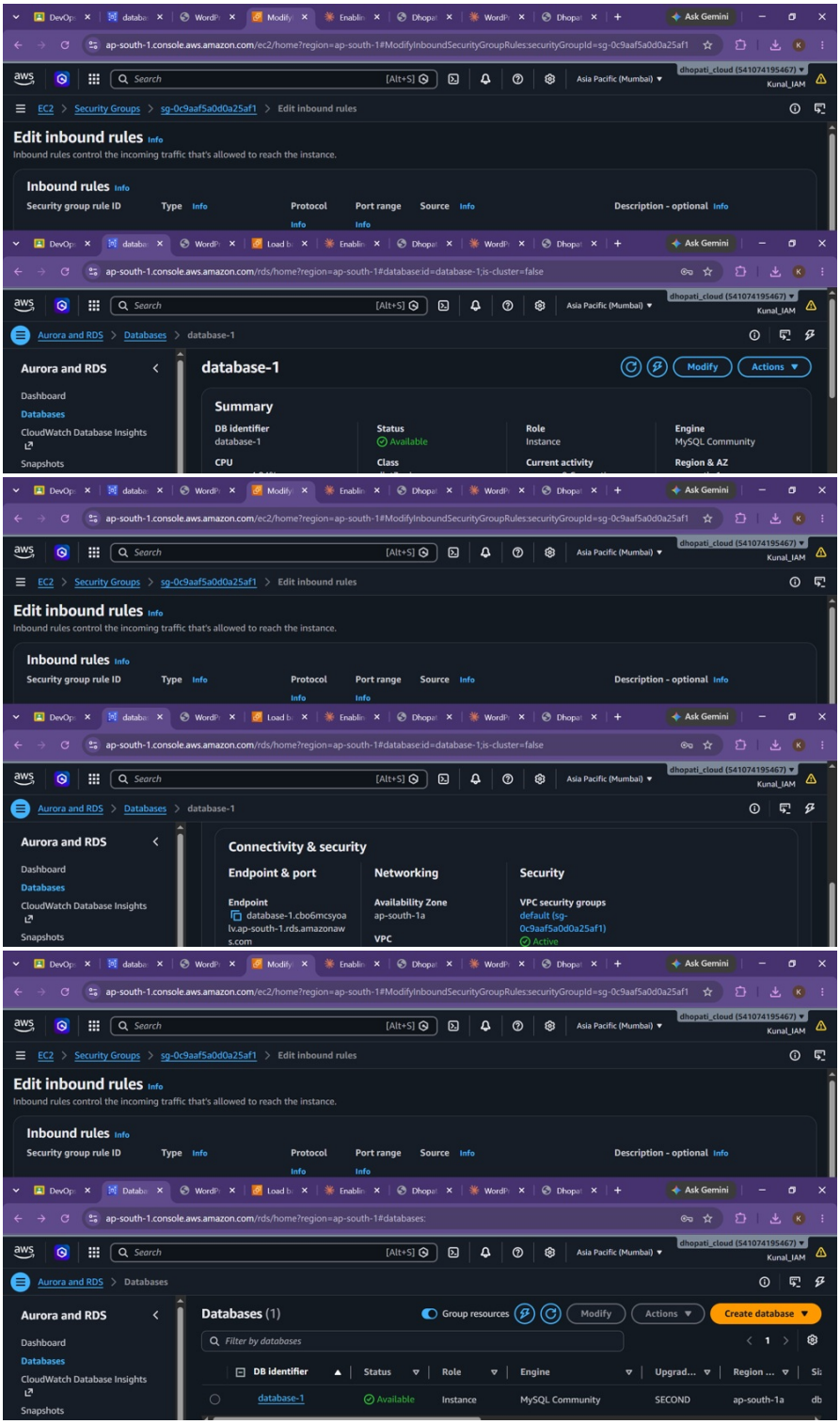
---

3. Database (RDS)

Step 2: Create RDS MySQL Database

- Created an **Amazon RDS MySQL** instance (`database-1`).
- **Public Access: No** — kept the database private, accessible only from within the VPC.
- Updated the RDS Security Group to allow **MySQL/Aurora (port**

3306) traffic from the EC2 instance's security group.



Step 3: Install MySQL Client on EC2

```
dnf install -y mariadb105
```

Export the RDS endpoint as an environment variable:

```
export MYSQL_HOST=database-1.cbo6mcsyoalv.ap-south-1.rds.amazonaws.com
```

Step 4: Connect to RDS and Create Database

```
mysql -h $MYSQL_HOST -P 3306 -u admin -p
```

Inside the MySQL shell:

```
CREATE DATABASE wordpress;
CREATE USER 'wpuser' IDENTIFIED BY 'admin123456789!';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'%';
FLUSH PRIVILEGES;
EXIT;
```

> ⚠ **Security Note:** Use a strong, unique password in production. Avoid hardcoding credentials directly in scripts or documentation.

---

4. Web Server & WordPress Setup

Step 5: Install and Configure Apache

```
sudo yum install -y httpd
sudo service httpd start
```

Step 6: Download and Extract WordPress

```
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
cd wordpress
```

Step 7: Configure `wp-config.php`

```
cp wp-config-sample.php wp-config.php
vi wp-config.php
```

Update the database connection details:

```
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wpuser' );
define( 'DB_PASSWORD', 'admin123456789' );
define( 'DB_HOST', 'database-1.cbo6mcsyoalv.ap-south-1.rds.amazonaws.com' );
```

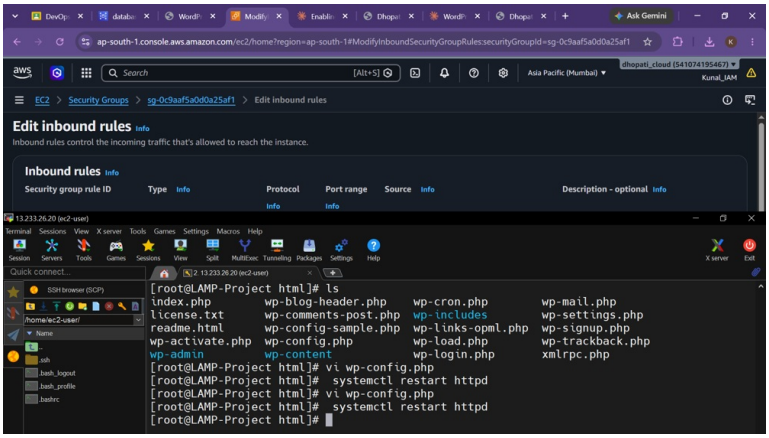
Generate unique security keys and salts from: `https://api.wordpress.org/secret-key/1.1/salt/`, and paste them in place of the default `AUTH_KEY`, `SECURE_AUTH_KEY`, `LOGGED_IN_KEY`, `NONCE_KEY`, and corresponding `*_SALT` values.

Step 8: Install PHP and Required Extensions

```
dnf install -y php8.4 php-mysqlnd
```

Step 9: Deploy WordPress Files

```
cd /home/ec2-user
sudo cp -r wordpress/* /var/www/html/
sudo service httpd restart
```



Terminal — WordPress Deployment

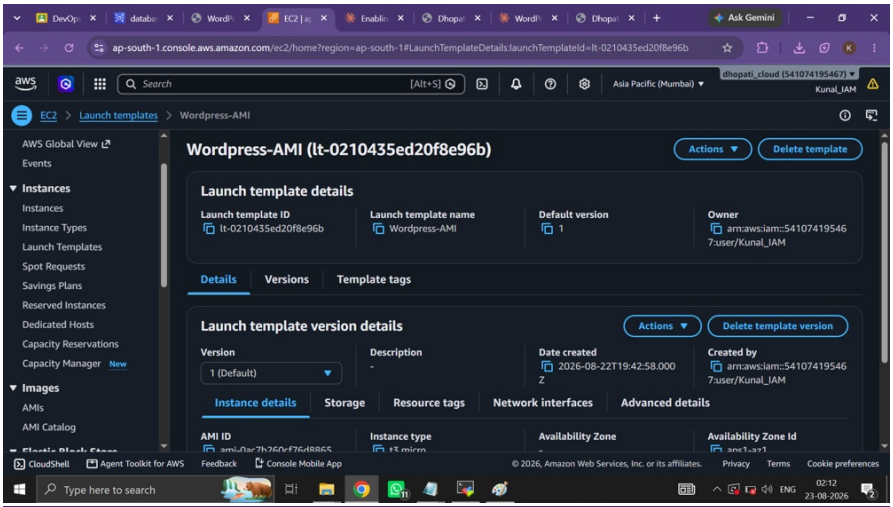
---

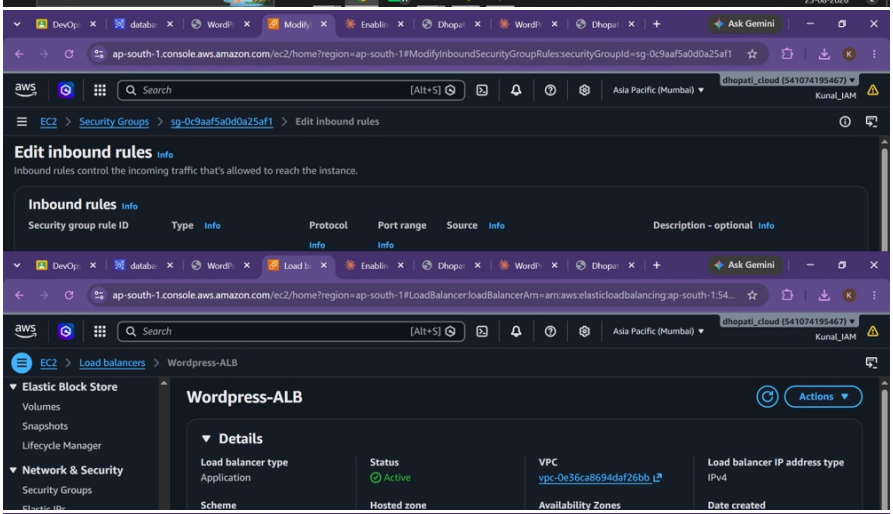
5. High Availability Extension: Blue-Green Deployment

To make the architecture scalable and highly available:

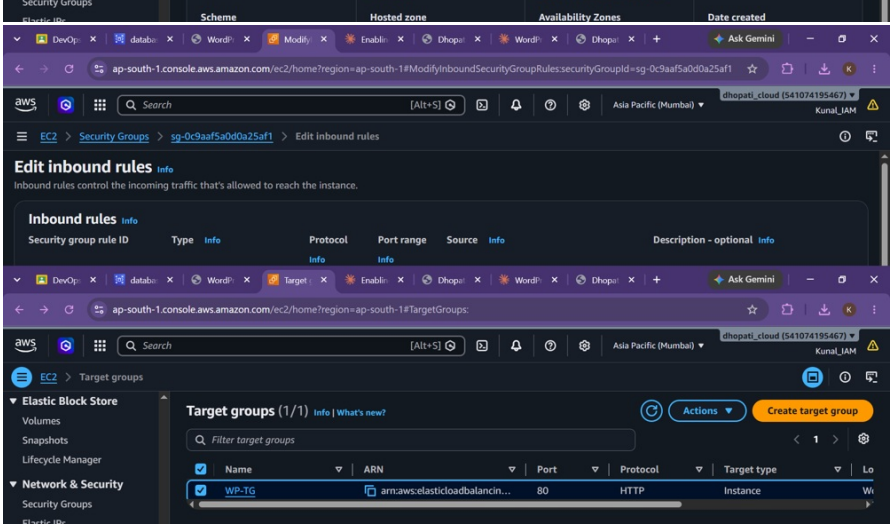
1. **Create an AMI** of the fully configured WordPress EC2 instance (`WordpressAMI`).
2. **Launch a second EC2 instance** from this AMI (skips manual DB setup steps since config is baked in).
3. **Create a Target Group** (`WP-TG`) — health checks against `/readme.html` or `index.php`.
4. **Create an Application Load Balancer** (`Wordpress-ALB`) and attach the Target Group.
5. **Create a Launch Template** using `WordpressAMI`.
6. **Create an Auto Scaling Group** (`wordpress-ASG`) using the Launch Template, attached to the Target Group.
7. **Access the application via the ALB DNS name** — traffic is now distributed and can scale automatically.

This setup supports **Blue-Green deployment**: new versions can be rolled out on a fresh AMI/target group, tested, and traffic can be shifted over via the ALB with minimal downtime.

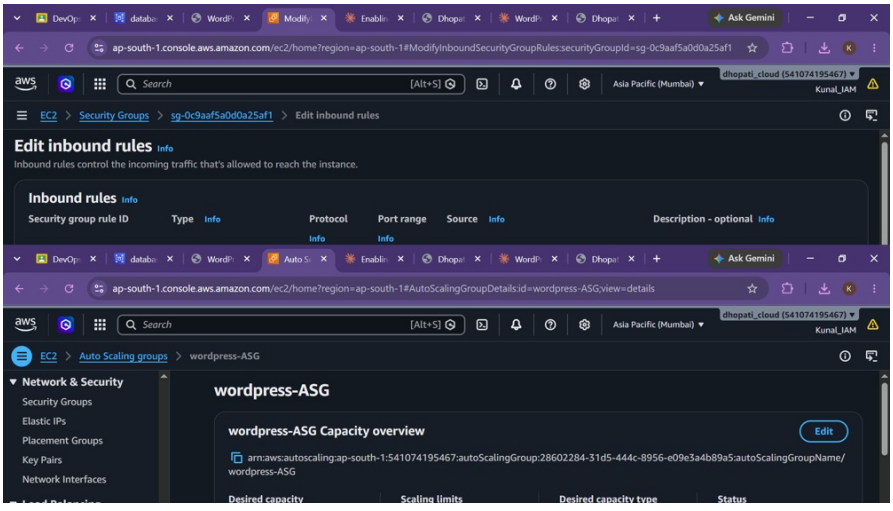

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#LaunchTemplateDetails:launchTemplateId=lt-0210435ed20f8e96b
EC2 > Launch templates > Wordpress-AMI
Wordpress-AMI (lt-0210435ed20f8e96b)
Actions
Delete template
Launch template details
Launch template ID
lt-0210435ed20f8e96b
Launch template name
Wordpress-AMI
Default version
1
Owner
arn:aws:iam::541074195467:user/Kunal_IAM
Details
Versions
Template tags
Launch template version details
Actions
Delete template version
Version
1 (Default)
Description
Date created
2026-08-22T19:42:58.000Z
Created by
arn:aws:iam::541074195467:user/Kunal_IAM
Instance details
Storage
Resource tags
Network interfaces
Advanced details
AMI ID
Instance type
Availability Zone
Availability Zone Id


ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#ModifyInboundSecurityGroupRules:securityGroupId=sg-0c9aaf5a0d0a25af1
EC2 > Security Groups > sg-0c9aaf5a0d0a25af1 > Edit inbound rules
Edit inbound rules
Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules
Security group rule ID
Type
Protocol
Port range
Source
Description - optional

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#LoadBalancer:loadBalancerArn=arn:aws:elasticloadbalancing:ap-south-1:54...
EC2 > Load balancers > Wordpress-ALB
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Wordpress-ALB
Actions
Details
Load balancer type
Application
Status
Active
VPC
vpc-0e36ca8694daf26bb
Load balancer IP address type
IPv4
Scheme
Hosted zone
Availability Zones
Date created


ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#ModifyInboundSecurityGroupRules:securityGroupId=sg-0c9aaf5a0d0a25af1
EC2 > Security Groups > sg-0c9aaf5a0d0a25af1 > Edit inbound rules
Edit inbound rules
Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules
Security group rule ID
Type
Protocol
Port range
Source
Description - optional

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#TargetGroups:
EC2 > Target groups
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Target groups (1/1)
Info | What's new?
Actions
Create target group
Filter target groups
Name
ARN
Port
Protocol
Target type
WP-TG
arn:aws:elasticloadbalancin...
80
HTTP
Instance


ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#ModifyInboundSecurityGroupRules:securityGroupId=sg-0c9aaf5a0d0a25af1
EC2 > Security Groups > sg-0c9aaf5a0d0a25af1 > Edit inbound rules
Edit inbound rules
Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules
Security group rule ID
Type
Protocol
Port range
Source
Description - optional

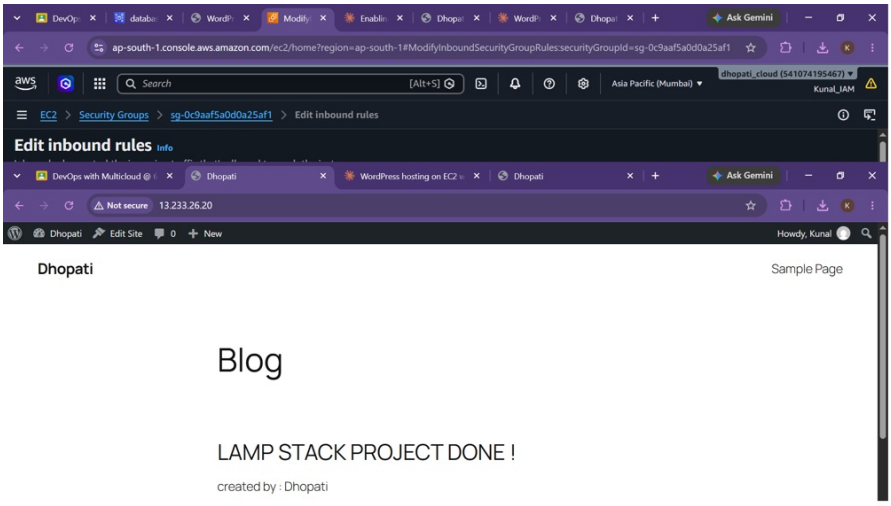
ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#AutoScalingGroupDetails:id=wordpress-ASG;view=details
EC2 > Auto Scaling groups > wordpress-ASG
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
wordpress-ASG
wordpress-ASG Capacity overview
Edit
arn:aws:autoscaling:ap-south-1:541074195467:autoScalingGroup:28602284-31d5-444c-8956-e09e3a4b89a5:autoScalingGroupName/wordpress-ASG
Desired capacity
Scaling limits
Desired capacity type
Status

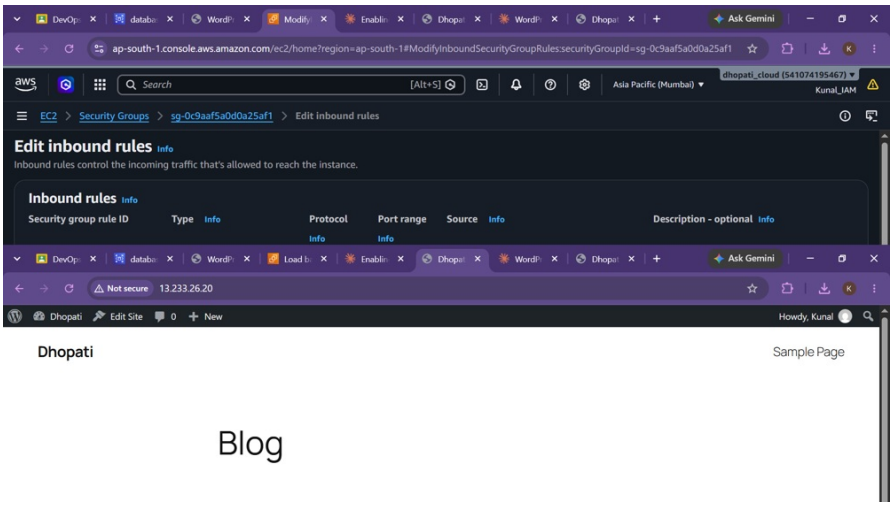
6. Live Verification

Step 10: Complete Installation via Browser

- `http://13.233.26.20/` → completed the WordPress setup wizard.
- `http://13.233.26.20/wp-admin` → WordPress admin dashboard login (verified working, logged in as Kunal).
- `http://Wordpress-ALB-1048355765.ap-south-1.elb.amazonaws.com` → WordPress ALB DNS link.

Test blog post **"LAMP STACK PROJECT DONE!"** was published and confirmed visible on the live site, proving the full Web → App → DB flow works end-to-end.





7. Known Limitations / Notes

Item	Status	Note
Security Group rules	Open (0.0.0.0/0) for HTTP, SSH, MySQL	Acceptable for a learning/demo project; should be tightened (specific IPs / internal SG only) for production
		EC2 was accessed directly via public IP

Bastion Host	Not used	+ SSH; can be added later for stricter access control
HTTPS/SSL	Not configured in this round	Site currently served over HTTP; ACM + HTTPS listener on the ALB would be the next step

---

Key Learnings

- Decoupling the database (RDS) from the application server (EC2) improves security, scalability, and maintainability.
- Keeping RDS **Publicly Inaccessible** and restricting access via Security Groups follows AWS security best practices.
- Using an AMI + Launch Template + ASG + ALB combination enables a horizontally scalable, fault-tolerant WordPress deployment.
- WordPress salts/keys should always be uniquely generated per deployment for session security.

Tech Stack Summary

`Amazon Linux 2023` · `Apache (httpd)` · `PHP 8.4` · `Amazon RDS (MySQL)` · `WordPress` · `Application Load Balancer` · `Auto Scaling Group` · `AMI`

Project Status

End-to-end functional — WordPress is live and publicly reachable both directly via EC2 and through the Application Load Balancer, backed by a private RDS MySQL database, with AMI + ASG in place for horizontal scaling and Blue-Green style deployments.